



भारतीय सूचना प्रौद्योगिकी संस्थान गुवाहाटी
Indian Institute of Information Technology Guwahati
CS236 - ARTIFICIAL INTELLIGENCE LAB
SOLUTIONS - 01

1. Compose a Python program that accomplishes the following:
 - i. Assign a value to a variable and display both the value and its data type.
 - ii. Assign values to two variables and execute these arithmetic operations: addition, subtraction, multiplication, and division.
 - iii. Assign a string to a variable and determine the string's length.
 - iv. Assign a string value to a variable, and then convert the variable's type to an integer.

```
# 1. Assign a value to a variable and display both the value and its data type.

my_variable = 42 # You can change this to any value
print("Value:", my_variable)
print("Data Type:", type(my_variable))

# 2. Assign values to two variables and execute arithmetic operations.

num1 = 10
num2 = 5

addition = num1 + num2
subtraction = num1 - num2
multiplication = num1 * num2
division = num1 / num2

print("\nArithmetic Operations:")
print("Addition:", addition)
print("Subtraction:", subtraction)
print("Multiplication:", multiplication)
print("Division:", division)

# 3. Assign a string to a variable and determine the string's length.

my_string = "Hello, world!"
string_length = len(my_string)

print("\nString Operations:")
print("String:", my_string)
print("String Length:", string_length)

# 4. Assign a string value to a variable, and then convert the variable's type to an integer.
# Note: The string must contain a valid integer representation for this to work without errors.

string_number = "123"
try:
    integer_number = int(string_number)
    print("\nType Conversion:")
```

```

print("Original String:", string_number)
print("Converted Integer:", integer_number)
print("Converted Integer Data Type:", type(integer_number))
except ValueError:
    print("\nError: The string cannot be converted to an integer.")

```

2. Develop a Python program to interchange the values of two variables, both with and without utilizing a temporary variable.

```

# Method 1: Using a temporary variable

def swap_with_temp(a, b):
    """Swaps the values of two variables using a temporary variable.

    Args:
        a: The first variable.
        b: The second variable.
    """
    temp = a # Store the value of 'a' in a temporary variable
    a = b    # Assign the value of 'b' to 'a'
    b = temp # Assign the value stored in 'temp' (original 'a') to 'b'
    print("After swapping (with temp):")
    print("a =", a)
    print("b =", b)

# Example usage:
x = 10
y = 5
print("Before swapping:")
print("x =", x)
print("y =", y)
swap_with_temp(x, y)

# Method 2: Without using a temporary variable (using simultaneous assignment)

def swap_without_temp(a, b):
    """Swaps the values of two variables without using a temporary variable.

    Args:
        a: The first variable.
        b: The second variable.
    """
    a, b = b, a # Simultaneously assign 'b' to 'a' and 'a' to 'b'
    print("\nAfter swapping (without temp):")
    print("a =", a)
    print("b =", b)

# Example usage:
x = 10
y = 5
print("\nBefore swapping:")
print("x =", x)
print("y =", y)
swap_without_temp(x, y)

# Method 3: Using addition and subtraction (for numeric variables)

def swap_arithmetic(a, b):
    """Swaps the values of two numeric variables using addition and subtraction.

    Args:

```

```

        a: The first variable
        b: The second variable
    """
    a = a + b
    b = a - b
    a = a - b
    print("\nAfter swapping (using arithmetic):")
    print("a =", a)
    print("b =", b)

# Example usage
x = 10
y = 5
print("\nBefore swapping:")
print("x =", x)
print("y =", y)
swap_arithmetic(x,y)

# Method 4: Using XOR (for numeric variables)
def swap_xor(a, b):
    """Swaps the values of two numeric variables using XOR.

    Args:
        a: The first variable
        b: The second variable
    """
    a = a ^ b
    b = a ^ b
    a = a ^ b
    print("\nAfter swapping (using XOR):")
    print("a =", a)
    print("b =", b)

# Example usage:
x = 10
y = 5
print("\nBefore swapping:")
print("x =", x)
print("y =", y)
swap_xor(x,y)

```

3. Given the string `str = "I am a computer science student"`, perform the following:

- i. Display the initial character of the string.
- ii. Display the characters from the second to the fifth position.
- iii. Display all characters starting from the third position.

```

str = "I am a computer science student"

# 1. Display the initial character of the string.
initial_char = str[0] # Strings are indexed starting from 0
print("Initial character:", initial_char)

# 2. Display the characters from the second to the fifth position.

second_to_fifth = str[1:5] # Slice from index 1 (second char) up to (but not including) index 5
print("Characters from second to fifth position:", second_to_fifth)

```

```
# 3. Display all characters starting from the third position.

from_third = str[2:] # Slice from index 2 (third char) to the end of the string
print("Characters from the third position onwards:", from_third)
```

4. Create a Python program that does the following:

- i. Generate a list containing six elements and display the entire list.
- ii. Display the elements of the list from the third position to the end of the list.
- iii. Insert the element 40 into the middle of the list, and append the element 100 to the end of the list.
- iv. Display the third, fourth, and fifth elements of the list.

```
# 1. Generate a list containing six elements and display the entire list.

my_list = [10, 20, 30, 5, 15, 25] # Example list with six elements
print("Original list:", my_list)

# 2. Display the elements of the list from the third position to the end of the list.

third_to_end = my_list[2:] # Slice from index 2 (third element) to the end
print("Elements from the third position to the end:", third_to_end)

# 3. Insert the element 40 into the middle of the list,
#    and append the element 100 to the end of the list.

list_length = len(my_list)
middle_index = list_length // 2 # Calculate the middle index (integer division)

my_list.insert(middle_index, 40) # Insert 40 at the middle index
my_list.append(100) # Append 100 to the end of the list

print("List after insertion and appending:", my_list)

# 4. Display the third, fourth, and fifth elements of the list.
# (after modifications in step 3)

third_fourth_fifth = my_list[2:5] # Slice from index 2 up to (not including) index 5
print("Third, fourth, and fifth elements:", third_fourth_fifth)
```

5. Write a program to combine two lists into a single list and display it. Then, print the even numbers present in the merged list.

```
def combine_and_print_even(list1, list2):
    """Combines two lists and prints the even numbers in the merged list.

    Args:
        list1: The first list.
        list2: The second list.
    """

    merged_list = list1 + list2 # Combine the lists using the + operator
    print("Merged list:", merged_list)

    print("Even numbers in the merged list:")
    for number in merged_list:
        if number % 2 == 0: # Check if the number is even
            print(number)
```

```
# Example usage:
list1 = [1, 3, 5, 7, 2]
list2 = [2, 4, 6, 8, 9]

combine_and_print_even(list1, list2)
```

6. Develop a Python program that performs the following:

- i. Create a tuple containing a mix of data types and display it.
- ii. Create a tuple that repeats the word programming three times.
- iii. Create two tuples and then concatenate them into a single tuple.

```
# 1. Create a tuple containing a mix of data types and display it.

mixed_tuple = (10, "hello", 3.14, True, [1, 2, 3])
print("Mixed tuple:", mixed_tuple)

# 2. Create a tuple that repeats the word "programming" three times.

repeated_tuple = ("programming",) * 3 # Note the comma after "programming" to make it a tuple
print("Repeated tuple:", repeated_tuple)

# 3. Create two tuples and then concatenate them into a single tuple.

tuple1 = (1, 2, 3)
tuple2 = (4, 5, 6)
concatenated_tuple = tuple1 + tuple2 # Use the + operator to concatenate tuples
print("Concatenated tuple:", concatenated_tuple)
```

7. Develop a Python program that does the following:

- i. Given the dictionary person = {'first_name': 'John', 'last_name': 'Doe', 'age': 25, 'favorite_colors': ['blue', 'green'], 'active': True}, display the dictionary.
- ii. Display the keys and values of the dictionary separately.
- iii. Display the value associated with the second key.

```
person = {
    'first_name': 'John', 'last_name': 'Doe', 'age': 25,
    'favorite_colors': ['blue', 'green'], 'active': True
}

# 1. Display the dictionary.

print("Dictionary:", person)

# 2. Display the keys and values of the dictionary separately.

print("Keys:", person.keys())
print("Values:", person.values())

# 3. Display the value associated with the second key.
# Note: Dictionaries are unordered, so "second key" is not guaranteed.
# We'll assume it means the second key in the order it was defined (for this example).

keys = list(person.keys()) # Convert keys to a list to access by index
if len(keys) >= 2:
    second_key = keys[1]
    second_key_value = person[second_key]
```

```

    print("Value associated with the second key ('" + second_key + "):", second_key_value)
else:
    print("The dictionary does not have at least two keys.")

```

```

from collections import OrderedDict

```

```

# Define the dictionary using OrderedDict for deterministic ordering of keys

```

```

person = OrderedDict({
    'first_name': 'John',
    'last_name': 'Doe',
    'age': 25,
    'favorite_colors': ['blue', 'green'],
    'active': True
})

```

```

# 1. Display the dictionary

```

```

print("Dictionary:", person)

```

```

# 2. Display the keys and values of the dictionary separately

```

```

print("Keys:", list(person.keys()))

```

```

print("Values:", list(person.values()))

```

```

# 3. Display the value associated with the second key

```

```

# Since we're using OrderedDict, the order of insertion is guaranteed

```

```

second_key = list(person.keys())[1] # Access the second key

```

```

second_key_value = person[second_key]

```

```

print(f"Value associated with the second key ('{second_key}'): {second_key_value}")

```

8. Write a Python program that accepts a number as input and determines whether it is even or odd using an if-else statement.

```

# Get input from the user
number = int(input("Enter a number: "))

```

```

# Check if the number is even or odd

```

```

if number % 2 == 0:
    print(f"{number} is even.")

```

```

else:

```

```

    print(f"{number} is odd.")

```

9. Construct a program to display numbers from a list using loops.

```

def display_numbers_from_list(numbers):
    """Displays each number in a list using a loop.

```

```

    Args:

```

```

        numbers: A list of numbers.

```

```

    """

```

```

    for number in numbers:

```

```

        print(number)

```

```

# Example list:

```

```

my_list = [1, 5, 10, 15, 20, 25]

```

```

# Call the function to display the numbers:

```

```

display_numbers_from_list(my_list)

```

10. Create a Python function that accepts a list and returns a new list containing only the unique elements of the original list.

```
def get_unique_elements(input_list):
    """
    Returns a new list containing only the unique elements of the input list,
    preserving the original order.

    Args:
        input_list: The original list.

    Returns:
        A new list with unique elements.
    """
    unique_list = []
    seen = set() # Use a set to keep track of elements we've already encountered

    for element in input_list:
        if element not in seen:
            unique_list.append(element)
            seen.add(element)

    return unique_list

# Example usage
my_list = [1, 2, 2, 3, 4, 4, 5, 1, 6, 3]
unique_list = get_unique_elements(my_list)
print("Original list:", my_list)
print("List with unique elements:", unique_list)
```

11. Create a dictionary `student_grades` where keys are student names (strings) and values are their corresponding grades (integers). Write a program to:
 - i. Print the grade of a specific student.
 - ii. Add a new student and their grade to the dictionary.
 - iii. Update the grade of an existing student.
 - iv. Iterate through the dictionary and print the names of students who scored above a certain threshold (e.g., 90).

```
student_grades = {
    "Alice": 95,
    "Bob": 85,
    "Charlie": 92,
    "David": 78,
    "Eve": 88
}

# 1. Print the grade of a specific student.

student_name = "Bob"
if student_name in student_grades:
    grade = student_grades[student_name]
    print(f"{student_name}'s grade: {grade}")
else:
    print(f"Student {student_name} not found in the dictionary.")

# 2. Add a new student and their grade to the dictionary.

new_student = "Frank"
new_grade = 93
student_grades[new_student] = new_grade
print(f"Added {new_student} with grade {new_grade}.")
```

```

# 3. Update the grade of an existing student.

student_to_update = "Eve"
updated_grade = 90
if student_to_update in student_grades:
    student_grades[student_to_update] = updated_grade
    print(f"Updated {student_to_update}'s grade to {updated_grade}.")
else:
    print(f"Student {student_to_update} not found in the dictionary.")

# 4. Iterate through the dictionary and print the names of students
# who scored above a certain threshold (e.g., 90).

threshold = 90
print(f"Students who scored above {threshold}:")
for name, grade in student_grades.items():
    if grade > threshold:
        print(name)

# Print the updated dictionary (optional)
print("\nUpdated dictionary:", student_grades)

```

12. Given a dictionary inventory representing items and their quantities in a store, write a program that uses dictionary methods (`keys()`, `values()`, `items()`, `get()`) to:

- i. Display all the items in the inventory.
- ii. Display the quantities of each item.
- iii. Check if a specific item is in stock and print its quantity (or a “Not found” message).

```

inventory = {
    "apple": 10,
    "banana": 15,
    "orange": 5,
    "grape": 20,
    "mango": 8
}

# 1. Display all the items in the inventory.

print("Items in inventory:")
for item in inventory.keys():
    print(item)

# 2. Display the quantities of each item.

print("\nQuantities of each item:")
for quantity in inventory.values():
    print(quantity)

# 3. Check if a specific item is in stock and print its quantity (or a "Not found" message).

item_to_check = "banana"
quantity = inventory.get(item_to_check) # Use get() to avoid KeyError

if quantity is not None:
    print(f"\nQuantity of {item_to_check}: {quantity}")
else:
    print(f"\n{item_to_check} is not found in the inventory.")

```



```

item_to_check = "kiwi"
quantity = inventory.get(item_to_check)

if quantity is not None:
    print(f"\nQuantity of {item_to_check}: {quantity}")
else:
    print(f"\n{item_to_check} is not found in the inventory.")

# Alternative way to display items and quantities using items()
print("\nItems and Quantities (using items()):")
for item, quantity in inventory.items():
    print(f"{item}: {quantity}")

```

13. Write a function `calculate_rectangle_area` that takes length and width as parameters (with default values of 1) and returns the area of a rectangle. Demonstrate how to call the function:

- i. With only positional arguments.
- ii. With keyword arguments.
- iii. Using the default values.

```

def calculate_rectangle_area(length=1, width=1):
    """Calculates the area of a rectangle.

    Args:
        length: The length of the rectangle (default: 1).
        width: The width of the rectangle (default: 1).

    Returns:
        The area of the rectangle.
    """
    return length * width

# 1. Calling with only positional arguments:
area1 = calculate_rectangle_area(5, 10) # length=5, width=10
print("Area 1 (positional arguments):", area1)

# 2. Calling with keyword arguments:
area2 = calculate_rectangle_area(length=3, width=7) # Explicitly specify length and width
print("Area 2 (keyword arguments):", area2)

area3 = calculate_rectangle_area(width=4, length=6) # Order doesn't matter with keyword arguments
print("Area 3 (keyword arguments, different order):", area3)

# 3. Calling using the default values:
area4 = calculate_rectangle_area() # No arguments provided, so length=1, width=1
print("Area 4 (default values):", area4)

# You can also mix positional and keyword arguments:
area5 = calculate_rectangle_area(5, width=2) # length=5 (positional), width=2 (keyword)
print("Area 5 (mixed arguments):", area5)

```

14. Write a program with a global variable and a function that defines a local variable with the same name. Demonstrate how the function accesses the local variable and the global variable can be accessed outside the function.

```

# Global variable
my_variable = 10

def my_function():

```

```

"""Demonstrates local and global variable scope."""
# Local variable with the same name as the global variable
my_variable = 5
print("Inside function: my_variable (local) =", my_variable)

# Call the function
my_function()

# Access the global variable outside the function
print("Outside function: my_variable (global) =", my_variable)

```

15. Write a function `process_student_data` that takes a list of dictionaries as input. Each dictionary represents a student and contains keys like `name`, `id`, and `grades` (where `grades` is a list of numbers). The function should:

- i. Calculate the average grade for each student.
- ii. Return a new dictionary where keys are student IDs and values are their average grades.

```

def process_student_data(student_data):
    """
    Calculates the average grade for each student and returns a dictionary
    mapping student IDs to their average grades.

    Args:
        student_data: A list of dictionaries, where each dictionary represents a
            student with keys 'name', 'id', and 'grades' (a list
            of numbers).

    Returns:
        A dictionary where keys are student IDs and values are their average
        grades.
    """
    student_averages = {}
    for student in student_data:
        student_id = student['id']
        grades = student['grades']
        average_grade = sum(grades) / len(grades) if grades else 0
        student_averages[student_id] = average_grade
    return student_averages

# Example Usage:
student_data = [
    {'name': 'Alice', 'id': 'A123', 'grades': [90, 85, 92, 88]},
    {'name': 'Bob', 'id': 'B456', 'grades': [78, 80, 75]},
    {'name': 'Charlie', 'id': 'C789', 'grades': [95, 92, 98, 90]},
    {'name': 'David', 'id': 'D101', 'grades': []} # Empty grades list
]

average_grades = process_student_data(student_data)
print(average_grades)

```